

Lucas Zacarias Martins Neves

**Detecção de vulnerabilidades de memória em C:
Abordagem integrando VSA e HOFG**

São Paulo

2025

Lucas Zacarias Martins Neves

Detecção de vulnerabilidades de memória em C: Abordagem integrando VSA e HOFG

Proposta de Trabalho de Conclusão de Curso
apresentado ao SENAC Santo Amaro como
requisito parcial para aprovação na disciplina
de TCC1 do curso de Ciência da computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Bacharelado em Ciência da computação

Orientador: Prof. Nome do Orientador

São Paulo
2025

Resumo

Palavras-chave: Vazamento de memória, HOFG.

Sumário

1	INTRODUÇÃO	4
1.1	Objetivos	4
1.1.1	Objetivo geral	4
1.1.2	Objetivos específicos	4
1.2	Justificativa	4
2	REVISÃO BIBLIOGRÁFICA	6
2.1	Referencial teórico	6
2.2	Metodologia da pesquisa bibliográfica	6
2.3	Métricas de Avaliação	6
2.4	Fundamentos	6
2.5	Trabalhos relacionados	6
3	DESENVOLVIMENTO	7
3.1	Solução proposta	7
3.2	Cronograma de trabalho	7

1 Introdução

1.1 Objetivos

1.1.1 Objetivo geral

O objetivo principal do trabalho é implementar um analisador estático de código para a linguagem C, com foco na detecção de vulnerabilidades de memória, através da integração da representação Heap Object Flow Graph (HOFG) com técnicas de Value-Set Analysis (VSA) para o rastreamento preciso de ponteiros (offset e range).

1.1.2 Objetivos específicos

1. Fundamentar teoricamente a integração do Value-Set analysis à estrutura do Heap Object Flow Graph (HOFG), demonstrando como essa abordagem conjunta pode mitigar as limitações de rastreamento de ponteiros e reduzir a taxa de falsos positivos da análise estática.
2. Implementar motor de análise estática processando a representação intermediária LLVM IR, construindo os algoritmos necessários para a extração e geração do HOFG e sua integração com o VSA.
3. Avaliar a precisão e a eficiência do analisador implementado utilizando conjuntos de testes que exploram falhas da linguagem C (como vazamentos de memória e usos após desalocação).
4. Comparar os resultados obtidos com referências comparativas tendo como foco principal a taxa de falsos positivos e verdadeiro positivos, e comparar o tempo e custo computacional da implementação original do HOFG, para validar o impacto da integração.

1.2 Justificativa

A linguagem C tem aplicações no desenvolvimento de sistemas operacionais, sistemas embarcados, drivers, softwares de alta performance, dentre outros. Sendo uma linguagem que permite o desenvolvedor gerenciar a alocação de itens na memória heap, sem a implementação nativa de elementos tiram tal responsabilidade dos desenvolvedores, encontra como problemas principais os riscos incluindo a má gestão de ponteiros e vazamentos de memória.

Mesmo com a existência de linguagens de programação como Rust, sendo focado em segurança, existem projetos que apresentariam custos excessivos para portabilidade entre as linguagens, fazendo com que seja necessário o uso da linguagem original do desenvolvimento, como o C, neste caso. Com isso, para garantir maior segurança de tais projetos, um analisador de código pode ser desenvolvido como uma das soluções, com foco em encontrar potenciais vazamentos e problemas provenientes da má gestão de itens armazenados na memória.

2 Revisão Bibliográfica

2.1 Referencial teórico

2.2 Metodologia da pesquisa bibliográfica

Design Science Research

2.3 Métricas de Avaliação

2.4 Fundamentos

2.5 Trabalhos relacionados

O analisador será avaliado a partir do Juliet Test Suite, sendo um conjunto de casos da linguagem C que exploram as suas vulnerabilidades. O resultado obtido será comparado com os resultados de analisadores e algoritmos desenvolvidos que possuem como finalidade o mesmo objetivo, ou seja, de garantir a segurança dos projetos. O principal analisador a ser comparado é o HOFG-Analyser, sendo a aplicação original da representação de programas.

3 Desenvolvimento

3.1 Solução proposta

3.2 Cronograma de trabalho